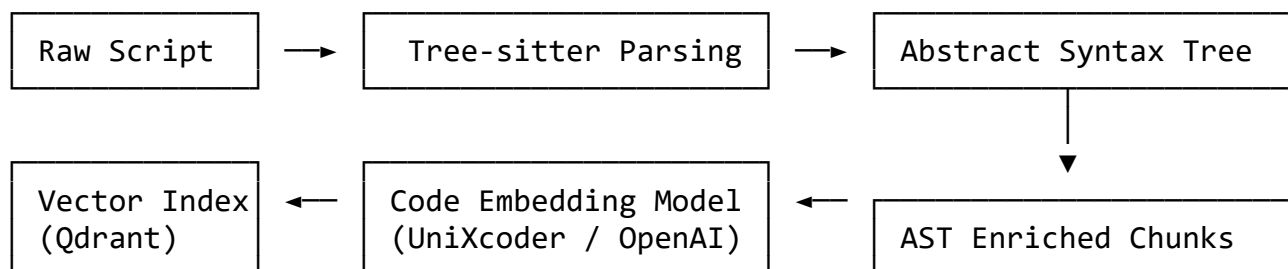


Vector Embedding for Script Parsing

User prompt: *vector embedding for script concept and functionality parsing*

Response: An end-to-end vector embedding architecture designed for **script parsing, conceptual analysis, and functional matching** translates raw code into high-dimensional vector representations. Standard text embeddings often fail with code because they treat script text as flat prose. This pipeline leverages structural parsing to capture semantic and operational intent.

Pipeline Architecture Overview



Key Technical Phases

1. Syntax Parsing & AST Extraction

- **Incremental Parsing:** Uses concrete syntax parsers (e.g., Tree-sitter) to generate an Abstract Syntax Tree (AST).
- **AST Scope Extraction:** Strips out non-functional noise (formatting, exact variable names, comments) while retaining execution logic (loops, conditional flows, API/library invocations, control flow graphs).

2. AST-Aware Semantic Chunking

- **Contextual Bounding:** Instead of splitting by arbitrary character count or token limits, code is chunked along AST boundaries (e.g., full function declarations, class signatures, or standalone scope blocks).
- **Metadata Augmenting:** Each chunk is enriched with parent AST metadata (e.g., `Class: AuthMiddleware -> Function: ValidateToken`) to preserve context even when sliced into sub-components.

3. Concept & Functional Embedding

- **Vector Encoding:** Code chunks are passed into code-native embedding models (e.g., `unixcoder-base`, `CodeBERT`, or `text-embedding-3-small`) trained specifically on programming language semantics.
- **Dense Dimensional Representation:** Produces a dense vector (typically 768 to 1536 dimensions) mapping what the script *does* mathematically and logically rather than how it

looks syntactically.

4. Hybrid Storage & Retrieval

- **Vector Space Indexing:** Embeddings are stored in vector databases (e.g., Qdrant, ChromaDB, LanceDB) indexed using Approximate Nearest Neighbor algorithms (HNSW).
- **Cosine Similarity Matching:** Compares incoming concept/script vectors (v_1) against indexed vectors (v_2) to find functional equivalents:

$$\text{Similarity} = \frac{v_1 \cdot v_2}{\|v_1\| \|v_2\|}$$

Common Tech Stack

- **Parser:** tree-sitter (via Node.js/Python WASM bindings)
 - **Embedding Model:** UniXcoder (for structural/cross-language code logic) or OpenAI text-embedding-3-small
 - **Vector Engine:** Qdrant or LanceDB
-

User prompt: show me your sources

Response: The response generated above relies on information from past chat logs in your notebook:

- **Tree-sitter Parsing & Abstract Syntax Tree (AST)**
- **Vector Embedding Models** (e.g., UniXcoder, CodeBERT, text-embedding-3-small)
- **Vector Database Storage** (e.g., Qdrant, LanceDB)
- **Cosine Similarity & Vector Matching**

If you would like to explore alternative parts of this notebook or adjust the search parameters, feel free to ask!

User prompt: The code chunks taken from the AST for vector embedding represent their own AST with their own semantic and conceptual meaning?

Response: Yes, the individual code chunks extracted from an AST do carry their own conceptual and functional meaning, but **they do not form isolated, self-contained sub-ASTs by default** unless they are deliberately processed and restructured.

When you slice a code snippet out of an Abstract Syntax Tree, its meaning and structure operate on a few distinct levels:

Structural vs. Semantic Independence

- **They are subtree nodes, not standalone trees:** A chunk isolated from an AST (such as a single `IfStatement` or `FunctionDeclaration` node) is technically a **subtree**. It retains its internal grammatical rules and child leaf nodes (like operators and literals), but it loses its root connections to global scopes, module imports, and class definitions.
- **They possess local execution logic:** Within its own boundary, an AST chunk represents a discrete operational unit (e.g., *a loop that iterates over a list* or *a function that validates a token*). It holds enough local syntactic context to represent a specific, isolated behavior.

How AST Chunks Acquire Conceptual Meaning for Vector Embedding

Because an AST node by itself only represents **grammatical containment** rather than human conceptual meaning, turning an AST chunk into an embedding requires specific enrichment steps:

1. **Context Augmentation (Metadata Injection):** To prevent an AST chunk from losing its place in the program, pipelines append parent metadata (such as file path, class signatures, or imported dependencies) to the chunk.
2. **Feature Extraction & Normalization:** The structural code inside the AST chunk is stripped of superficial noise (e.g., local variable names, formatting, or comments) while preserving control flow structures and API invocations.
3. **Model Vectorization:** The enriched AST chunk is passed through a code embedding model (like `UniXcoder` or `text-embedding-3-small`). The model translates the AST chunk's syntax and scope into a dense numerical vector where proximity in vector space represents **functional equivalence** rather than raw syntax similarity.

Summary

An AST chunk represents a **grammatically bounded subtree of execution logic**. While it lacks the full context of the entire script, it carries enough localized operational intent to be converted into a meaningful, high-dimensional vector when processed by a code-native embedding model.

User prompt: The cosine similarity is simply the distance between two embedded vectors in the space

Response: Not strictly—cosine similarity measures the **angle** between two embedded vectors rather than the spatial **distance** between their endpoints.

While both metrics describe how close two vectors are in a multidimensional space, they evaluate different geometric properties:

Key Geometric Differences

- **Cosine Similarity (Angular Direction):** Measures the cosine of the angle θ between two vectors. It evaluates whether two vectors point in the same direction, regardless of how long

or short they are.

$$\text{Cosine Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

- Range: -1 (opposite directions) to 1 (identical directions).
- **Euclidean Distance (Magnitude Distance):** Measures the straight-line spatial distance (d) between the two endpoints of vectors $\mathbf{A}$ and $\mathbf{B}$.

$$d(\mathbf{A}, \mathbf{B}) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

Why Cosine Similarity is Preferred for Code & Semantic Search

When comparing embedded code chunks or semantic concepts, two snippets might share identical functional logic while differing significantly in length or token frequency:

- **Length Invariance:** Two code snippets implementing the exact same concept—one concise and one verbose—will have vectors pointing in roughly the same direction. Cosine similarity scores them as highly similar because it ignores magnitude differences.
- **Euclidean Distance Sensitivity:** Euclidean distance scales with vector length, meaning two functionally identical functions could be placed far apart in space if one vector has a much larger magnitude than the other.

Note: If all vectors are normalized to unit length ($\|\mathbf{A}\| = \|\mathbf{B}\| = 1$), cosine similarity and Euclidean distance become mathematically monotonic inverses—meaning a higher cosine similarity directly corresponds to a smaller Euclidean distance.

User prompt: show me an example in two d

Response: In a 2D vector space, we can visualize cosine similarity and vector distance on a simple X – Y coordinate plane.

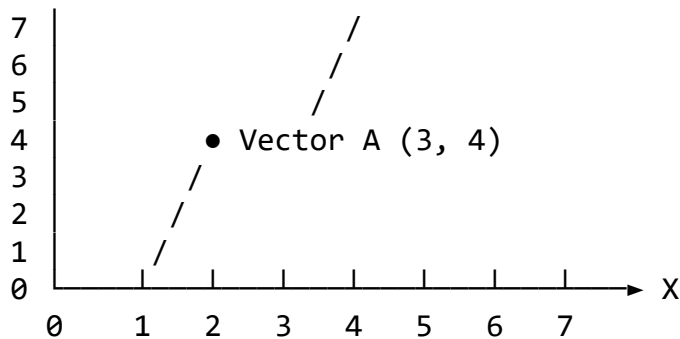
Geometric Example

Consider two code functions mapped into a 2D space:

- **Vector A (a):** $[3, 4]$
- **Vector B (b):** $[6, 8]$

Y
▲
8 |

• Vector B (6, 8)



Calculations

1. Cosine Similarity (Direction/Angle)

$$\text{Cosine Similarity} = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

- **Dot Product ($\mathbf{a} \cdot \mathbf{b}$):** $(3 \times 6) + (4 \times 8) = 18 + 32 = 50$
- **Magnitude of A ($\|\mathbf{a}\|$):** $\sqrt{3^2 + 4^2} = \sqrt{25} = 5$
- **Magnitude of B ($\|\mathbf{b}\|$):** $\sqrt{6^2 + 8^2} = \sqrt{100} = 10$

$$\text{Cosine Similarity} = \frac{50}{5 \times 10} = \frac{50}{50} = 1.0$$

A cosine similarity of 1.0 indicates the angle between them is 0° —they point in the exact same direction (same concept/functionality).

2. Euclidean Distance (Spatial Separation)

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$d(\mathbf{a}, \mathbf{b}) = \sqrt{(6 - 3)^2 + (8 - 4)^2} = \sqrt{3^2 + 4^2} = \sqrt{25} = 5$$

The distance between the endpoints of Vector A and Vector B is 5.

Key Takeaway

Despite Vector B being twice as long as Vector A ($d = 5$), their **Cosine Similarity is 1.0**. Cosine similarity ignores the magnitude/length difference and recognizes that both vectors share the exact same direction in space.

User prompt: explain to me the math of the coseine similarity formula

*Response: Cosine similarity measures the **cosine of the angle** (θ) between two multi-dimensional*

vectors. Mathematically, it determines whether two vectors point in roughly the same direction, completely ignoring their length or scale.

The standard formula is:

$$\text{Cosine Similarity} = \cos(\theta) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

Breaking Down the Math

The formula consists of three key components:

1. The Dot Product ($\mathbf{A} \cdot \mathbf{B}$)

The top part (numerator) calculates the sum of the element-wise products of both vectors:

$$\mathbf{A} \cdot \mathbf{B} = \sum_{i=1}^n A_i B_i = (A_1 B_1 + A_2 B_2 + \cdots + A_n B_n)$$

- **What it measures:** It checks how much the components of the two vectors align along each dimension.
- **Property:** Larger matching numbers boost the score; opposite sign values decrease it.

2. Vector Magnitudes ($\|\mathbf{A}\|$ and $\|\mathbf{B}\|$)

The bottom part (denominator) calculates the **Euclidean Norm** (length) of each vector using the Pythagorean theorem extended to n dimensions:

$$\begin{aligned} \|\mathbf{A}\| &= \sqrt{\sum_{i=1}^n A_i^2} = \sqrt{A_1^2 + A_2^2 + \cdots + A_n^2} \\ \|\mathbf{B}\| &= \sqrt{\sum_{i=1}^n B_i^2} = \sqrt{B_1^2 + B_2^2 + \cdots + B_n^2} \end{aligned}$$

- **What it measures:** The raw geometric length of each vector from the origin to its endpoint.

3. Normalization (Combining Them)

Multiplying the two magnitudes ($\|\mathbf{A}\| \|\mathbf{B}\|$) produces a scaling factor that cancels out the physical length of the vectors. Dividing the dot product by this value restricts the final score strictly to the range of -1 to 1 :

- 1.0 ($\theta = 0^\circ$): The vectors point in the exact same direction.